Online Assessment Archive

Software Development Engineer (SDE)

Squarepoint Capital
IIT Kharagpur

Section 1: Mathematics & Algorithms

1. Modulo Sum Function

NUMBER THEORY

Given a function **C(i, j)** defined for integers *i* and *j* as:

$$C(i, j) = (i \bmod j) + (j \bmod i)$$

Find the value of the function **f(n)** for a given input *n*, where **f(n)** is defined as:

$$f(n) = \Sigma_{(i=1 \text{ to } n)} \Sigma_{(j=1 \text{ to } n)} C(i, j)$$

Constraints: $n \leq 10^6$

SOLUTION INSIGHT

Because *C(i, j)* is perfectly symmetric, the sum simplifies significantly. The total sum is equivalent to calculating $2 \times \Sigma (i \bmod j)$ for all pairs up to *n*. For a fixed *j*, the sum $\Sigma (i \bmod j)$ is a known sequence that can be computed using Harmonic Lemma or direct summation in $O(n \log n)$ time, easily fitting the 10^6 constraint.

2. Tree Traversal Integrity

GRAPHS / TREES

The candidate is asked to verify if a given sequence of vertices is a valid Breadth-First Search (BFS) traversal of a given tree. This problem tests graph representations and BFS queue state management.

Platform Match:

Codeforces 1037/D - Valid BFS?

Section 2: Dynamic Programming & Game Theory

3. Two Cores and a Latch (Optimal Operation Split)

DP / GAME THEORY

Two cores process operations in a strictly given order. You are provided an array `time` where `time[i]` is the time needed to finish the ith operation.

A "latch" mechanism decides which core runs each operation. The core currently holding the latch has a choice: it can assign the current operation to itself, which causes the latch to pass to the other core; or it can assign the operation to the other core, keeping the latch for itself. Each core plays optimally to maximize its own total running time. Core 1 holds the latch initially.

Return the final answer as an array consisting of the maximum time taken by core 1 and core 2: `[total_time_core1, total_time_core2]`.

EXAMPLE 1

Input: `[141, 592, 653]`

Output: `[733, 653]`

Core 1 gives away 141 and keeps the latch. It then runs 592 itself. The latch passes to Core 2, which runs 653. Core 1 gets 141+592 = 733.

EXAMPLE 2

Input: `[10, 21, 10, 21, 10]`

Output: `[41, 31]`

Core 1 optimally maximizes its score over the series of alternating high and low values, resulting in 41 vs 31.

Exact Matches & Similar Platform Problems:

This logic mirrors the classic DP minimax choice where a player either takes the current item and yields control, or skips and retains control.

- Codeforces 859C - Pie Rules (Exact Logic Match)
- LeetCode 877 - Stone Game (Conceptual Variation)
- LeetCode 1140 - Stone Game II (Conceptual Variation)